Tabular Q-Learning versus Deep Q-Networks on Gymnasium Control Tasks: Discretization, Hyperparameter Sensitivity, and Ablations

José Reinoso
School of Mathematical and Computational Sciences
Yachay Tech University, Urcuquí, Ecuador

Aldrin Chavez
School of Mathematical and Computational Sciences
Yachay Tech University, Urcuquí, Ecuador

***Abstract*—We compare tabular reinforcement learning with Deep Q-Networks (DQN) on Gymnasium environments under a common protocol: equal budgets of environment steps, several independent seeds, and greedy evaluation reported separately from training returns. On CartPole-v1, tabular Q-learning with uniform state discretization (6, 10 and 20 bins per dimension and three velocity ranges) and SARSA are compared with DQN under the same 200k-step budget and five seeds. On LunarLander-v3 we study the sensitivity of DQN to the learning rate and to the target-network update period, ablate the target network and the replay buffer, and compare Double and Dueling DQN with vanilla DQN, with three seeds per configuration. On CartPole, discretized Q-learning reaches a return of about 200 within 20k steps but never solves the task, whereas DQN reaches the 475 threshold in all five seeds (median 90k steps) at 18 times the wall-clock cost per step, and then degrades to a final return of 159. On LunarLander, the default DQN ends at 207 ± 30; removing the replay buffer (-76) hurts more than removing the target network (83), and the largest learning rate makes one seed diverge. Double and Dueling DQN (214–223) stay within the across-seed spread of vanilla DQN. A written analysis quantifies, through action- and state-space counts, why continuous-action locomotion tasks and Atari pixel observations are out of reach for tabular methods and DQN, and which algorithm families suit them.**

***Index Terms*—reinforcement learning, Q-learning, SARSA, deep Q-network, discretization, ablation study**

I. Introduction

Value-based reinforcement learning (RL) estimates the action-value function $Q(s,a)$ and acts greedily with respect to it. In its tabular form, Q-learning [1] stores one value per state–action pair, which requires a finite and small state space [2]. Most control problems, however, have continuous observations. Two routes are then available: discretize the state space so that a table can still be used, or replace the table with a function approximator, as Deep Q-Networks (DQN) do with a neural network, an experience-replay buffer and a target network [3]. Discretization is simple and cheap per step, but the number of states grows exponentially with the number of dimensions [4]. DQN generalizes across states, but bootstrapping with a nonlinear approximator trained off-policy can be unstable [2].

This paper compares both routes on Gymnasium [5] tasks under a common protocol that follows the recommendations of [6]: equal budgets of environment steps, multiple seeds, and greedy evaluation separated from training returns. Our contributions are:

- a comparison of tabular Q-learning (three bin counts, three velocity ranges, and SARSA) with DQN on CartPole-v1 under the same 200k-step budget (Tier 1);
- a one-at-a-time study of the learning rate and the target-update period of DQN on LunarLander-v3, with ablations of the target network and of the replay buffer (Tier 2);
- a comparison of Double [7] and Dueling [8] DQN with vanilla DQN on the same task (Tier 3, option B);
- a quantitative analysis of why continuous-action locomotion and pixel-based Atari games fall outside the reach of tabular methods and DQN.

Section II justifies the choice of environments, Sections III and IV describe the methods and the protocol, Sections V and VI report and discuss the results, and Section VII contains the analysis of unsuitable environments. Code, configurations and logs are available at https://github.com/gato09sa/rl-dqn-project01.

TABLE I
Candidate environments (Gymnasium 1.3.0). "10-bin $|\mathcal{S}|$": number of table states with 10 bins per continuous dimension.

Environment	Obs.	$\lvert\mathcal{A}\rvert$	Threshold	10-bin $\lvert\mathcal{S}\rvert$	Suits
CartPole-v1	4	2	475	10^4	tabular, DQN
MountainCar-v0	2	3	-110	10^2	tabular, DQN
Acrobot-v1	6	3	-100	10^6	DQN
LunarLander-v3	8†	4	200	4×10^6	DQN

†Six continuous dimensions and two binary leg-contact flags.

II. Pertinence Analysis

A task suits tabular RL if its state space is finite and small, or can be discretized into a table whose entries are visited often enough within the training budget. It suits DQN if its action space is discrete, because the greedy action is an arg max over one network output per action; the observation space may be continuous. Table I applies these criteria to the candidate tasks.

CartPole-v1 has four continuous observation dimensions, two of which (cart and pole velocities) are unbounded, so bin ranges must be chosen by hand. With 10 bins per dimension the table has 10^4 states and 2×10^4 entries, ten times fewer than

the 2×10^5 training steps, so tabular learning is feasible. With 20 bins it has 1.6×10^5 states and 3.2×10^5 entries, more than the step budget, so many entries can be updated only rarely, if at all. The two actions are discrete, so DQN applies directly. CartPole therefore suits both families and is our Tier 1 task.

LunarLander-v3 has six continuous dimensions, two binary flags and four discrete actions. Ten bins per continuous dimension give 4×10^6 states and 1.6×10^7 entries, 40 times the 400k-step budget: the table cannot be filled, whereas DQN only needs the discrete action space. It is therefore a DQN task, used for Tiers 2 and 3. MountainCar-v0 (Tier 1 alternative) and Acrobot-v1 (Tier 2 alternative) were not used. Under MountainCar's sparse reward a naive DQN rarely reaches the goal, so the comparison would measure exploration rather than discretization; on CartPole both families can reach the threshold. Acrobot targets weak hardware, but a timing test on one of our laptops (i5-10210U, four parallel runs) showed LunarLander fits the CPU budget.

TABLE II
HYPERPARAMETERS OF THE DEFAULT CONFIGURATIONS.

	Tier 1 (CartPole)		Tiers 2–3
Parameter	Tabular	DQN	DQN (LunarLander)
Step budget / seeds	200k / 5	200k / 5	400k / 3
Evaluation period	5k	5k	10k
Discount γ	0.99	0.99	0.99
ε: start $\rightarrow$ end	$1 \rightarrow 0.05$	$1 \rightarrow 0.05$	$1 \rightarrow 0.05$
Decay steps T_ε	100k	20k	100k
Step size α	0.1	–	–
Bins per dimension	10 (6, 20)	–	–
Hidden layers	–	128, 128	256, 256
Learning rate	–	5×10^{-4}	5×10^{-4} [a]
Batch size B	–	64	128
Replay capacity N	–	50k	100k
Warm-up steps T_0	–	1k	10k
Update every k steps	1	1	4
Target period C (steps)	–	1000	1000 [b]

Also studied: [a] 10^{-4} and 2×10^{-3}; [b] 250 and 4000 steps.

III. Methods

A. Tabular Q-learning and SARSA

Both methods update a table $Q(s, a)$ after every transition $(s_t, a_t, r_{t+1}, s_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[y_t - Q(s_t, a_t) \right], \tag{1}$$

with target $y_t = r_{t+1} + \gamma \max_a Q(s_{t+1}, a)$ for Q-learning (off-policy) [1] and $y_t = r_{t+1} + \gamma\, Q(s_{t+1}, a_{t+1})$ for SARSA (on-policy), where a_{t+1} is the action the agent actually takes next [9]. If s_{t+1} is terminal, $y_t = r_{t+1}$. An episode cut by the time limit is *not* terminal, so its last transition still bootstraps from s_{t+1} [10]. The table is initialized to zero, actions are chosen ε-greedily, and ties in the greedy action are broken at random so that unvisited states do not bias the policy toward action 0.

B. State discretization

Each observation dimension i is split into b_i equal-width bins over a range $[l_i, h_i]$; values outside the range fall into the first or last bin. The per-dimension indices $\phi_i \in \{0, \ldots, b_i - 1\}$ are combined into a single state index $s = \sum_i \phi_i \prod_{j>i} b_j$, so $|\mathcal{S}| = \prod_i b_i$. For CartPole, the cart position and pole angle use their termination limits (± 2.4 and ± 0.21 rad); the unbounded velocities use ± 3.0 and ± 3.5 by default, ± 1.5 and ± 2.0 in the *narrow* variant, and ± 6.0 and ± 7.0 in the *wide* variant.

C. Deep Q-Network

A multilayer perceptron with ReLU activations maps an observation to one value per action, $Q(s, \cdot; \theta)$. Transitions are stored in a FIFO replay buffer $\mathcal{D}$ of capacity N; after T_0 warm-up steps, one gradient step is taken every k environment steps on a minibatch of size B sampled uniformly from $\mathcal{D}$, minimizing

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}} \Big[\ell_H \big(y - Q(s, a; \theta) \big) \Big], \tag{2}$$

$$y = r + \gamma\, (1 - d) \max_{a'} Q(s', a'; \theta^-), \tag{3}$$

where ℓ_H is the Huber loss [11], d is 1 only for true terminal states, and θ^- are the parameters of a target network copied from θ every C environment steps. We use Adam [12] with the gradient norm clipped at 10. The exploration rate ε decays linearly from 1.0 to 0.05 over T_ε steps and then stays constant. All networks are implemented in PyTorch [13].

D. Ablations

No target network: $\theta^- \equiv \theta$, i.e., the bootstrap target in (3) is computed by the online network itself (without gradient). *No replay buffer*: the buffer holds only the last B transitions and each update uses exactly this window, so minibatches are consecutive, correlated samples; the number of gradient steps is the same as in vanilla DQN.

E. Double and Dueling DQN (Tier 3)

Double DQN [7] decouples action selection from evaluation to reduce the overestimation caused by the max operator: $y = r + \gamma(1-d)\, Q\big(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-\big)$. Dueling DQN [8] splits the last layer into a state value and action advantages, $Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')$; subtracting the mean advantage makes V and A identifiable.

IV. Experimental Setup

Experiments. Tier 1 (CartPole-v1, 200k steps, 5 seeds) trains seven variants: Q-learning with 6, 10 and 20 bins, Q-learning with 10 bins and narrow or wide velocity ranges, SARSA with 10 bins, and DQN. Tier 2 (LunarLander-v3, 400k steps, 3 seeds) trains the default DQN, two alternative learning rates (10^{-4}, 2×10^{-3}), two alternative target-update periods ($C = 250$, 4000), and the two ablations, changing one value at a time. Tier 3 trains Double, Dueling and Double+Dueling DQN with the Tier 2 defaults and seeds, and reuses the Tier 2 default runs as the vanilla baseline. Table II lists all hyperparameters.

Evaluation protocol. Every 5k (Tier 1) or 10k (Tiers 2–3) environment steps the current greedy policy ($\varepsilon = 0$) is evaluated on 10 episodes with fixed environment seeds;

training returns are logged separately, one value per episode. After training, the final greedy policy is evaluated on 100 episodes with unseen seeds, which matches the definition of the success thresholds (mean return over 100 episodes). *Steps to threshold* is the first periodic evaluation whose 10-episode mean reaches the threshold. All curves use environment steps on the x-axis, so tabular agents and DQN are compared at equal experience. They show the mean over seeds with a band of ±1 population standard deviation; training curves are smoothed with a 10-episode moving average and interpolated onto a common step grid.

Exploration schedules. In Tier 1 the tabular agents decay ε over half the budget and DQN over 20k steps. A table has no generalization, so every state has to be visited; DQN generalizes and benefits from exploiting earlier. The step budget, which is the quantity compared, is identical.

Reproducibility and hardware. Python, NumPy, PyTorch and the environments are seeded per run. Runs execute as four parallel processes with one CPU thread each; every run stores its configuration, hardware and library versions. Tier 1 ran on an Intel Core i5-10210U with Python 3.14.4 and PyTorch 2.14.1, and Tiers 2–3 on an Intel Core i5-10300H with Python 3.11.2 and PyTorch 2.5.1; both CPUs have 4 cores and 8 threads, both machines used Gymnasium 1.3.0 and NumPy 2.4.6, and all networks ran on the CPU. All comparisons within a tier, including wall-clock times, come from a single machine. Tier 1 runs record commit `6398ca6` (tag `v1-experiments`); Tiers 2–3 ran from commit `a1c62ff`, which differs only in not recording the commit in the run metadata. One command per experiment reproduces every figure (see the repository README).

V. Results

Table III summarizes every variant. We report two success measures: *reached*, a 10-episode evaluation during training at or above the threshold, and *solved*, a final 100-episode mean at or above it, which is the official criterion.

TABLE III

Summary over seeds. Final: return of the final greedy policy over 100 episodes, mean ± s.d. across seeds. Solved: seeds whose final return reaches the threshold. Reached: seeds with a 10-episode evaluation at or above it during training; Steps: median first step at which this happens. Time: mean wall-clock per run.

Variant	Final return	Solved	Reached	Steps	Time (s)
Tier 1: CartPole-v1, threshold 475, 5 seeds					
Q-learning, 6 bins	216.9 ± 53.8	0/5	0/5	–	44
Q-learning, 10 bins	199.2 ± 62.0	0/5	0/5	–	50
Q-learning, 20 bins	163.6 ± 47.9	0/5	0/5	–	45
Q-learning, 10 narrow	280.2 ± 107.1	1/5	1/5	145k	46
Q-learning, 10 wide	175.5 ± 29.0	0/5	0/5	–	44
SARSA, 10 bins	208.1 ± 70.4	0/5	0/5	–	44
DQN	159.2 ± 68.1	0/5	5/5	90k	823
Tier 2: LunarLander-v3, threshold 200, 3 seeds					
DQN default	206.6 ± 30.2	2/3	3/3	220k	689
lr 10^{-4}	118.7 ± 90.7	0/3	2/3	230k	764
lr 2×10^{-3}	112.9 ± 205.1	2/3	3/3	80k	788
$C = 250$	152.1 ± 88.1	1/3	3/3	260k	680
$C = 4000$	233.7 ± 28.3	2/3	3/3	250k	663
No target network	82.7 ± 76.0	0/3	3/3	100k	669
No replay buffer	−76.4 ± 77.1	0/3	2/3	265k	656
Tier 3: LunarLander-v3, threshold 200, 3 seeds					
Double DQN	220.0 ± 53.9	2/3	3/3	180k	681
Dueling DQN	222.7 ± 47.2	2/3	3/3	230k	738
Double + Dueling	214.4 ± 49.3	2/3	3/3	230k	759

A. Tier 1: tabular RL versus DQN on CartPole-v1

Fig. 1 compares Q-learning with 6, 10 and 20 bins with DQN. Per environment step the tables learn first: with 10 bins the mean evaluation return is 208 at 20k steps, against 122 for DQN. All tabular variants then plateau between about 150 and 280 for the rest of the budget. DQN overtakes them after about 85k steps, scores 500 in at least one evaluation in every seed, and its mean evaluation return peaks at 426 at 130k steps. It does not hold this level: the band widens to more than ±150, and the final policies return only 159 ± 68, below every tabular variant. Per second the order reverses: a tabular run processes its 200k steps in 44–50 s (about 4 400 steps/s), DQN in 823 s (about 240 steps/s). The 10-bin table first reaches an evaluation return of 200 after a median of 6 s; DQN after 145 s, and 475 after 340–390 s. The training-return panel favours DQN early mainly because its ε reaches 0.05 at 20k steps, against 100k for the tables.

Fewer bins gave higher final returns: 217 ± 54 with 6 bins, 199 ± 62 with 10 and 164 ± 48 with 20; the ordering is monotonic but each gap is smaller than one s.d. At the end of training the agents had visited 28% of the 1 296 states with 6 bins, 11% of the 10^4 with 10 bins and 2.4% of the 1.6×10^5 with 20 bins, i.e., about 370, 1 060 and 3 800 states, which received on average 540, 190 and 50 updates each. Fig. 2 varies the velocity ranges at 10 bins. Narrow ranges give the best tabular result (280 ± 107), including the only tabular seed that solves the task (final return 493), and raise the visited fraction to 17%; wide ranges give 176 ± 29 and lower it to 5.8%. SARSA (208 ± 70) and Q-learning (199 ± 62) with 10 bins are indistinguishable, and their curves overlap throughout (figure in the repository).

B. Tier 2: DQN on LunarLander-v3

Fig. 3 shows the hyperparameter study. The default DQN reaches 200 in every seed (median 220k steps) and ends at 207 ± 30, with two of three seeds solving the task. With a learning rate of 2×10^{-3} learning is fastest (median 80k steps) and two seeds end at 257 and 259, among the best final returns of Tier 2, but the third collapses after 230k steps: its evaluation return stays between −420 and −900 from 260k steps on and its final return is −177. Its mean Q (the average of $Q(s, a; \theta)$ over the training minibatches) climbs from 58 to 252 within 70k steps and then swings down to −59, whereas it never exceeds 58 in the other two seeds. With 10^{-4} learning is slower and less reliable: one seed never exceeds −9, and the mean Q decays to about 3 by the end. The target period mainly affects stability: $C = 4000$ gives the highest and most consistent final return (234 ± 28) and the lowest, smoothest mean Q (at most

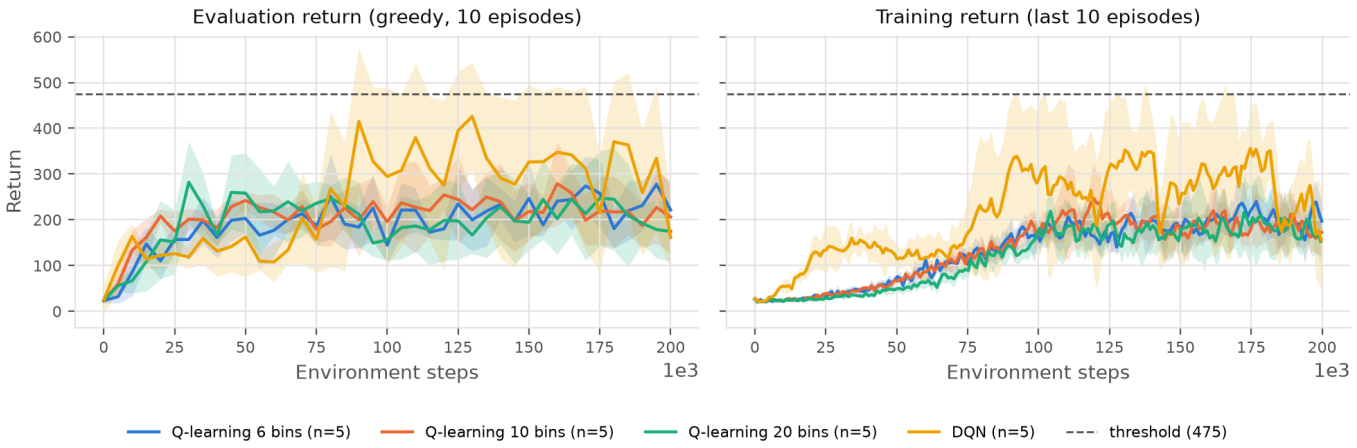


Fig. 1. Tier 1, CartPole-v1: tabular Q-learning (6, 10, 20 bins) versus DQN under the same 200k-step budget. Left: greedy evaluation return (10 episodes every 5k steps); right: training return. Mean over 5 seeds ±1 s.d.; dashed line: success threshold (475).

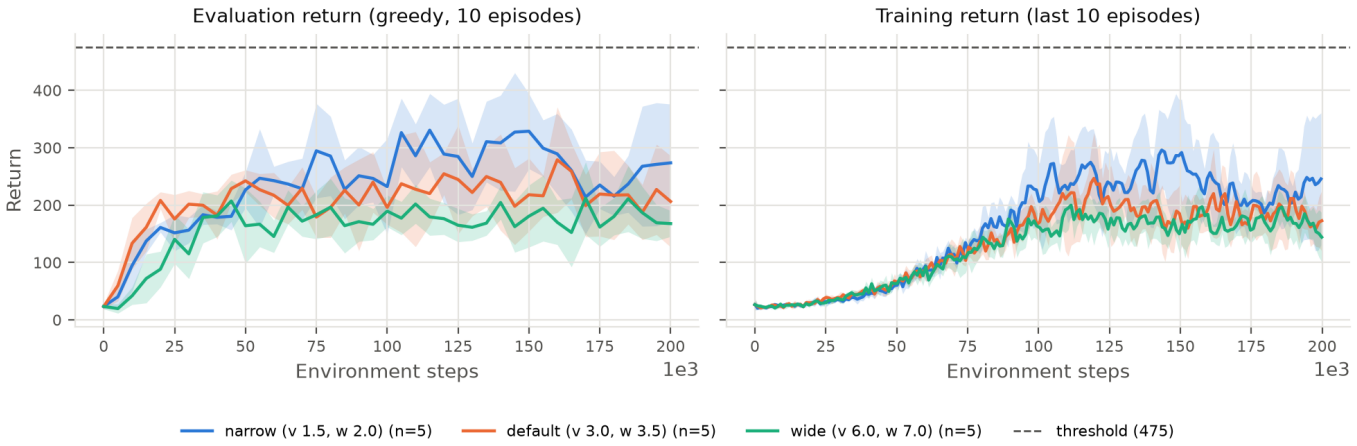


Fig. 2. Tier 1, CartPole-v1: Q-learning with 10 bins and three ranges for the cart velocity (v) and the pole angular velocity (w). Mean over 5 seeds ±1 s.d.

36), but reaches the threshold 30k steps later than the default; $C = 250$ ends at 152 ± 88, with higher mean Q (up to 64) and one seed that drops from 270 to 33 in its last evaluation.

Fig. 4 shows the ablation. Without a target network the agent reaches 200 in every seed, even earlier than full DQN (median 100k steps), but cannot keep it: its evaluations alternate between about 270 and 100 or less, no seed solves the task, and the final return is 83 ± 76. Its mean Q is erratic, from -26 at 20k steps to peaks of 121 and 155, against at most 56 for full DQN. Without a replay buffer, training returns stay below about 100, evaluations collapse suddenly (one seed scores $-3\,782$ at 370k steps), and the final return is -76 ± 77, the worst of all LunarLander runs.

C. Tier 3: Double and Dueling DQN

In Fig. 5 the four curves overlap within their bands. Double (220 ± 54), Dueling (223 ± 47) and Double+Dueling DQN (214 ± 49) end slightly above vanilla DQN (207 ± 30), and each solves the task in two of three seeds, as vanilla DQN does. Double DQN reaches the threshold first (median 180k steps, against 220k). The final mean Q is 32 for Double DQN and 35 for the other three variants.

VI. Discussion

A. Which method learned faster in Tier 1, and why?

The answer depends on the unit. Per second, the table wins by far: a Q-learning update reads and writes one entry, whereas DQN runs a forward and backward pass of the network on a minibatch of 64 at every step, about 18 times the cost per step on our CPU. Per environment step, the table is faster only at first. It shares nothing between states, but with 6 or 10 bins only 370–1 060 states are ever visited and each is updated hundreds of times, so the visited entries settle quickly. The table then stops improving: its plateau around 200 is reached by 20k steps and does not move during the remaining 180k, although ε keeps decreasing. This is a limit of the representation, not of the data. A grid cell merges states

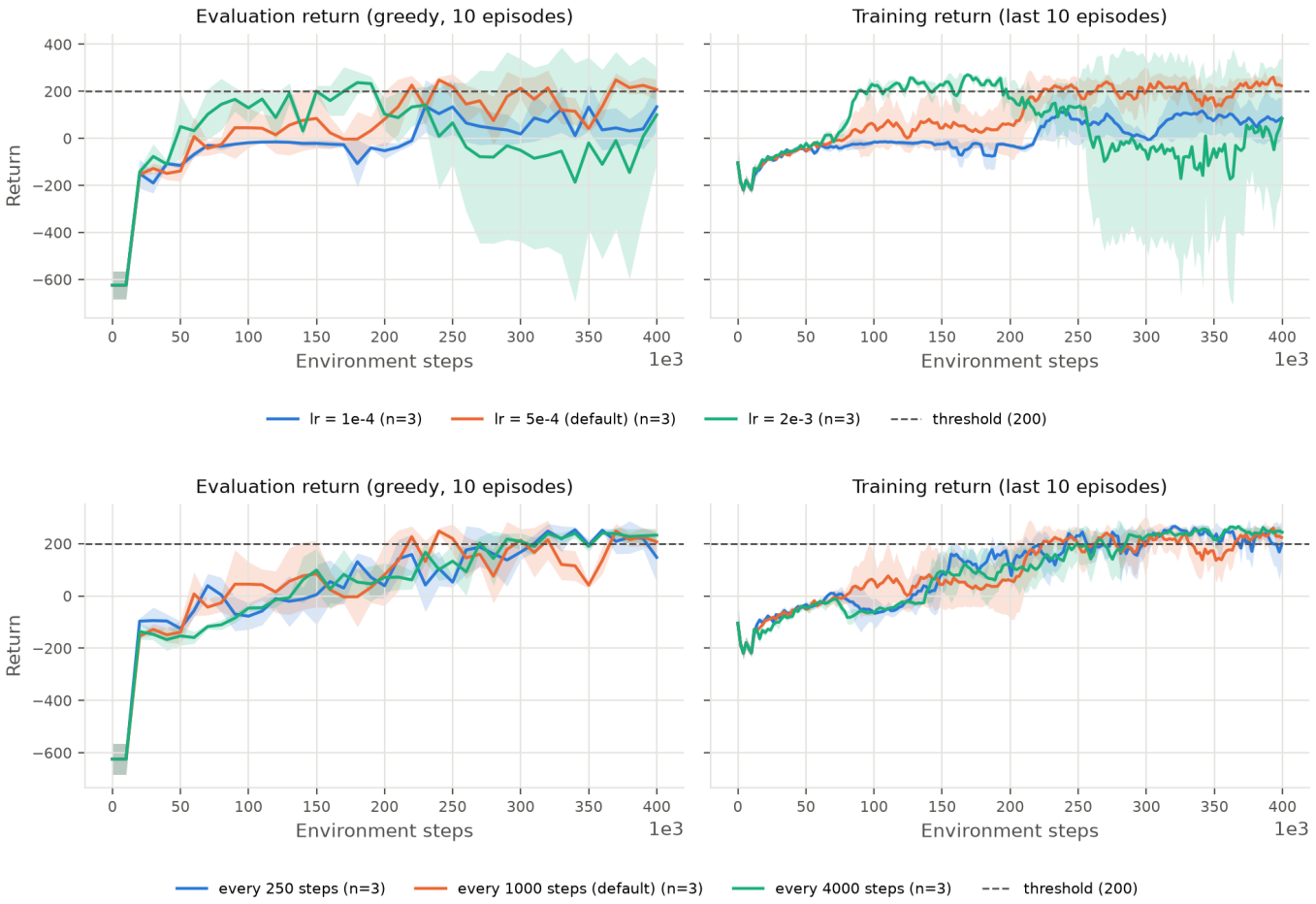


Fig. 3. Tier 2, LunarLander-v3: DQN sensitivity to the learning rate (top) and to the target-network update period C (bottom). Left: greedy evaluation return; right: training return. Mean over 3 seeds ±1 s.d.; dashed line: threshold (200).

whose best action differs, e.g., a pole leaning slightly left that is moving left or right within the same velocity bin, so the discretized problem is not Markov and no table over it balances for 500 steps reliably. DQN takes the continuous observation as input and generalizes across nearby states, so it can represent a finer decision boundary; fitting it takes more steps, but it is the only method that reaches 475.

DQN's late degradation is consistent with a known weakness of DQN on CartPole: once the policy balances for 500 steps, the 50k-transition buffer holds about 100 near-identical successful episodes, transitions close to failure are flushed out, and the network is trained only on states it already handles, so its values for the states that precede a fall drift. The logged mean Q is in line with this reading: it rises steadily to 89–96 at 200k steps, close to $1/(1-\gamma) = 100$, the value of a pole that never falls (time-limit cuts bootstrap). Because we evaluate the final policy, this degradation is counted; a best-checkpoint report would have hidden it.

B. *Sensitivity to the key design choice*

For the tables the key choice is the discretization, and two effects compete. More bins give finer resolution but more entries to fill: with 20 bins the 3 800 visited states receive about 50 updates each, against 540 with 6 bins, and with $\alpha = 0.1$ an entry needs tens of updates to forget its initial value and average its noisy targets, as Section II anticipated from the table size. The ranges matter as much as the count. With wide ranges most observed velocities fall into a few central bins, so only 5.8% of the table is used and the effective resolution is coarse; narrow ranges spend the 10 bins where the velocities actually are and clip only rare extreme values. What helps is resolution where the data lie, not the number of bins. SARSA and Q-learning coincide because their targets differ only in $Q(s', a')$ versus $\max_{a'} Q(s', a')$: once $\varepsilon = 0.05$, 95% of next actions are greedy, and CartPole has no state where one exploratory action is so costly that an on-policy learner would prefer a safer path, as in the cliff-walking example of [2].

For DQN the learning rate trades speed for stability. A step of 2×10^{-3} fits the targets faster, but it also moves the Q-function far between target syncs; in one seed the loop through



Fig. 4. Tier 2 ablation on LunarLander-v3: full DQN, without target network, and without replay buffer. The large negative excursions are single-seed collapses of the agent without replay. Mean over 3 seeds ±1 s.d.

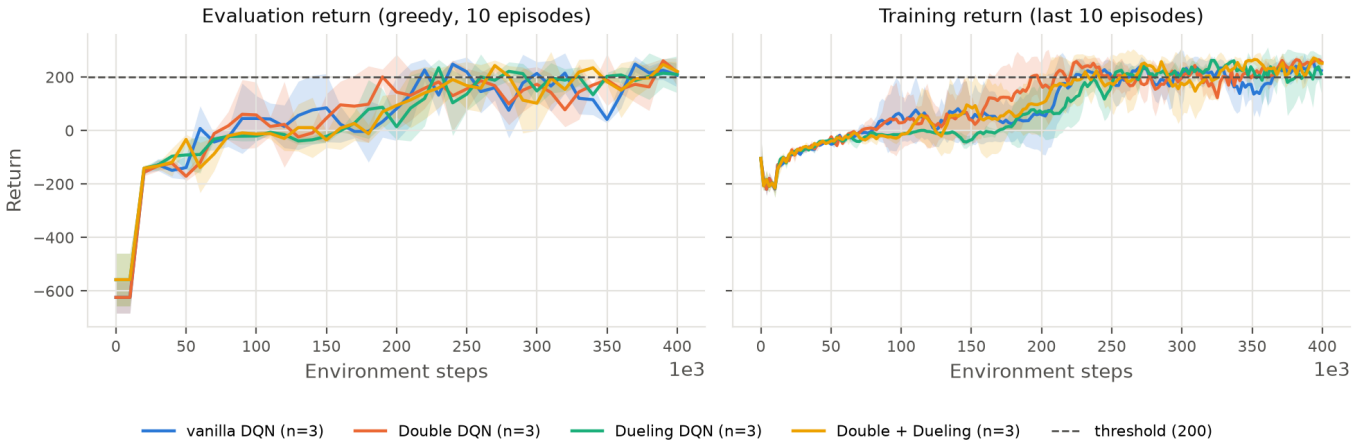


Fig. 5. Tier 3, LunarLander-v3: vanilla, Double, Dueling and Double+Dueling DQN. The vanilla runs are the Tier 2 default runs. Mean over 3 seeds ±1 s.d.

the bootstrap target amplified the values (mean Q up to 252) and the policy collapsed. A step of 10^{-4} was too small to learn within 400k steps in one seed. The target period acts in the opposite direction: a longer period keeps the regression target fixed for longer, which gave lower and smoother Q estimates and the most reliable final returns, at the cost of propagating value information more slowly. With three seeds, only the divergence at 2×10^{-3} is large relative to the across-seed spread.

C. What did the ablation reveal?

Both components matter, and in our setting the replay buffer matters more. Without a target network the bootstrap target $\max_{a'} Q(s', a'; \theta)$ changes with every gradient step, so each update partly chases its own output. Value information propagates immediately, which explains the early threshold crossings, but the estimates oscillate (mean Q from -26 to 155) and so does the policy. Without replay, every minibatch is the same window of the last 128 consecutive transitions, which are strongly correlated and come from one region of the state space. Gradient steps fit that region and overwrite what was learned elsewhere, and old transitions are never revisited, so the agent forgets, e.g., how to land while it is learning to hover; the result is the sudden collapses in Fig. 4 and the lowest final return. These are the two problems that replay and the target network were introduced to solve [3].

In Tier 3, Double DQN did not measurably reduce the mean Q (32 against 35) and no variant improved significantly on vanilla DQN. The overestimation that Double DQN removes grows with the noise of the estimates and with the number of actions in the max [7]; with four actions, a Huber loss, gradient clipping and a 1000-step target period, the bias of vanilla DQN appears to be small here. Dueling helps most when many actions have similar values [8], which LunarLander's four engines do not obviously provide. Detecting a gain of 10–15 points against a spread of 30–50 would require many more seeds.

TABLE IV

Action- and state-space sizes after uniform discretization, computed by instantiating each environment (Gymnasium 1.3.0, MuJoCo 3.17.0).

			$\lvert\mathcal{A}\rvert = k^d$		10-bin
Environment	Obs.	d	$k = 5$	$k = 11$	$\lvert\mathcal{S}\rvert$
BipedalWalker-v3	24	4	625	14 641	10^{24}
Hopper-v5	11	3	125	1 331	10^{11}
HalfCheetah-v5	17	6	15 625	1.8×10^{6}	10^{17}
Ant-v5	105†	8	3.9×10^{5}	2.1×10^{8}	10^{105}
Humanoid-v5	348	17	7.6×10^{11}	5.1×10^{17}	10^{348}

†Includes 78 contact-force terms; 27 without them.

D. Limitations

Tiers 2 and 3 use three seeds, so differences smaller than the across-seed spread are not conclusive. The hyperparameter study varies one value at a time around a single default and ignores interactions. Steps to threshold are measured with 10-episode evaluations, which are noisier than the 100-episode criterion. We report the final policy rather than the best checkpoint, so late performance drops are counted. Tier 1 and Tiers 2–3 ran on different machines and library versions, so wall-clock times are comparable only within a tier. The 10-episode *reached* criterion overstates success: without a target network every seed reached 200 but none solved the task. The mean Q is averaged over replayed state–action pairs and was not compared with Monte Carlo returns, so it shows trends but not the size of the overestimation bias. In Tier 1 the training returns of the two families are not directly comparable because their ε schedules differ.

VII. Analysis of Unsuitable Environments

A. Continuous-action locomotion

BipedalWalker-v3 and the MuJoCo [14] locomotion tasks have continuous, multidimensional action spaces (torques in $[-1, 1]^d$, $[-0.4, 0.4]^{17}$ for Humanoid) and observations of 11 to 348 dimensions (Table IV). DQN cannot use them directly: its greedy policy needs $\max_a Q(s, a)$, which over a continuous set is itself an optimization problem to solve at every step. Discretizing each of the d torque dimensions into k levels gives k^d joint actions, each with its own network output and its own value estimate. With $k = 11$ (a torque resolution of 0.2), HalfCheetah already needs 1.8×10^6 outputs and Ant 2.1×10^8; even $k = 5$ leaves Humanoid with 7.6×10^{11}. Most of these actions would never be tried, so their values would be pure extrapolation, and coarse levels cannot represent the smooth, coordinated torques of a gait. Factorizing the output per action dimension mitigates the explosion but changes the algorithm [15]. Tabular methods fail earlier, on the state side: 10 bins per dimension give 10^{24} states for BipedalWalker and 10^{105} for Ant, against typical budgets of about 10^6 environment steps.

B. Suitable algorithm families

These tasks call for actor-critic methods whose actor outputs a continuous action directly, which removes the maximization over actions, while a critic $Q(s, a)$ takes the action as an input and generalizes across it. *PPO* [16] is on-policy: a Gaussian policy is improved with a clipped surrogate objective that limits the size of each update. It is robust and parallelizes well across environments, but it discards data after each update, so it needs more samples. *SAC* [17] is off-policy: a stochastic actor maximizes return plus an entropy bonus that drives exploration, data are reused from a replay buffer, and two critics limit overestimation; it is among the most sample-efficient methods on MuJoCo. *TD3* [18] extends DDPG [19] with a deterministic actor, clipped double Q-learning, target-policy smoothing and delayed actor updates, which counter the same overestimation bias that Double DQN addresses in the discrete case. SAC and TD3 keep DQN's replay buffer and target networks; what changes is the actor that replaces the arg max.

TABLE V

Which family suits which problem structure.

States	Actions	Family	Example
Small or discretizable	Discrete	Q-learning, SARSA	CartPole
Continuous	Discrete	DQN and variants	LunarLander
Continuous	Continuous	PPO, SAC, TD3	Ant
Pixels	Discrete	DQN with a CNN	Pong

C. High-dimensional observations: Atari pixels

An observation of ALE/Pong-v5 [20] is a 210×160×3 array of 8-bit values, so there are up to $256^{100800} \approx 10^{242751}$ distinct observations; the usual preprocessing to four stacked 84×84 grayscale frames still leaves $256^{28224} \approx 10^{67970}$. A table over this space cannot be stored, and, more fundamentally, it cannot generalize: each entry is learned only from visits to that exact image, two frames that differ in one pixel are unrelated entries, and exact frames rarely repeat, so almost every entry would be visited at most once. Coarsening the pixels either keeps the dimensionality or erases the small objects (ball, paddles) that decide the game. A single frame is also not Markov, because it shows no velocity; this is why frames are stacked. DQN addresses both problems with a convolutional network whose shared weights map visually similar frames to similar values [3], at the cost of millions of frames and GPU time.

D. Summary: matching the problem to the algorithm family

Table V condenses the analysis. Each row adds one difficulty that breaks the method of the row above: a state space too large to enumerate breaks the table, a continuous action space breaks the arg max of DQN, and raw pixels require a network that extracts features from images before any value can be estimated.

VIII. AI-Use Statement

Generative AI was used under the course policy. A. Chavez used Claude (Anthropic) through Claude Code to explain the assignment, review the code for correctness, check the software environment, compute the environment specifications

in Tables I and IV, extract the statistics in Section V from the logged result files, and draft the text of this report. J. Reinoso used Claude (Anthropic) through Claude Code to set up the repository and environment, write the agents and the experiment scripts and configurations, integrate Weights & Biases logging, run smoke tests and timing measurements, and fill in parts of this report. Both authors reviewed all code and can explain every line of it. The authors verified all AI-generated content: environment dimensions were obtained by instantiating each environment, every reference was checked against its source [TODO: do it before submission], and every number in Section V comes from the logged result files. The most important prompts are listed in Appendix A.

References

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[3] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.

[5] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.

[7] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conference on Artificial Intelligence*, 2016.

[8] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. International Conference on Machine Learning (ICML)*, 2016.

[9] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Cambridge University Engineering Department, Tech. Rep. CUED/F-INFENG/TR 166, 1994.

[10] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proc. International Conference on Machine Learning (ICML)*, 2018.

[11] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[13] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[14] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[15] A. Tavakoli, F. Pardo, and P. Kormushev, "Action branching architectures for deep reinforcement learning," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[17] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. International Conference on Machine Learning (ICML)*, 2018.

[18] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. International Conference on Machine Learning (ICML)*, 2018.

[19] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Proc. International Conference on Learning Representations (ICLR)*, 2016.

[20] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013.

Appendix A
Most Important Prompts

Prompts that shaped the work, translated from Spanish and lightly edited for spelling (two consecutive messages are merged where one request spanned both), each followed by how its output was used and checked.

A. Chavez:

- "Explain everything this assignment says."
- "Review the files my teammate pushed and check whether my environment works for them."
- "Build the full report skeleton and fill in the parts that do not depend on results."

J. Reinoso:

1) *"Explain Tier 2 to me. Do you think it can run on my laptop?"*
   Use and verification: we timed 30k LunarLander steps with four parallel runs (about 73 s each), extrapolated about 20 min per 400k-step run, and kept LunarLander over Acrobot.
2) *"Aldrin gave me an idea so we do not step on each other's code: one of us takes care of the whole agents part, the tabular agent and the DQN for both tiers, and the other one launches the runs. What do you think, or what issues do you see? I liked it, but we are leaving out* `common` *and* `plot`*."*
   Use and verification: we adopted the split with a common agent interface (`act`, `greedy`, `update`); evaluation takes a policy function, so the same code evaluates the Q-table and the network.
3) *"Generate* `tabular.py` *(Q-learning and SARSA),* `discretizer.py` *and* `dqn.py` *with the flags* `use_target`, `use_replay`, `double` *and* `dueling`*. Keep in mind that we want it to run in parallel and use the cores; both PCs have 8 cores."*
   Use and verification: all 16 flag combinations and save/load were tested. The "8 cores" are 4 physical cores: 8 parallel runs took 284 s against 75 s for one, only 2× faster than sequential, so sweeps use 4 processes.
4) *"I am using CoreWeave [Forge, a Weights & Biases server]; I would like to upload the plots there."*
   Use and verification: CSV logs are the source of all figures and are mirrored to W&B; uploads were checked with test runs through the W&B API.
5) *"Aldrin says we should freeze the commits; it is for W&B. For that we need a line of code in each experiment, adjust it before running."*
   Use and verification: the logger records each run's commit, tag and uncommitted-change flag. All 35 Tier 1 runs report `v1-experiments` with no changes; the Tier 2–3 runs predate this, and their commit (`a1c62ff`) was recovered from W&B metadata.